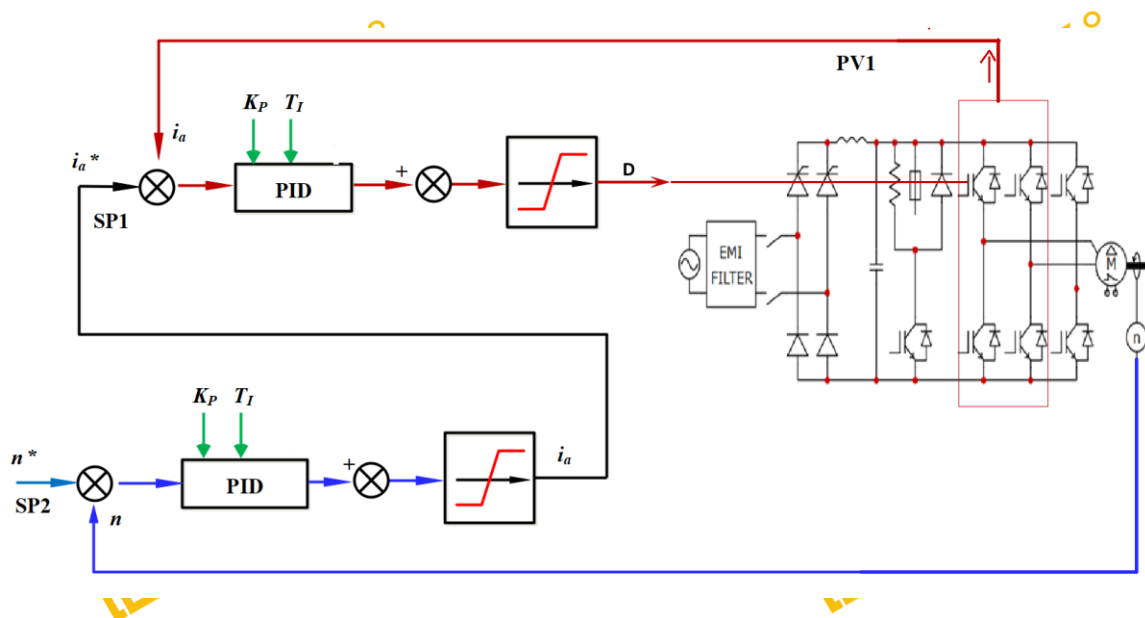


Tutorial – DC DRIVES WITH CASCADE CONTROL

Description

This tutorial guides you to demonstrate the way of driving a DC motor with the single-quadrant drive for dual closed loop speed control using the **DL 2106T06** module with bidirectional serial communication between a **Simulink-based Host Model** which will be running in the master (PC) and a **Target Model** running on **DL RCPCORE** module. The system implements basic control, monitoring, and fault protection based on real-time analog acquisition and serial communication.

As shown in Figure 1, in the inner loop, the controlled variable is the armature current (I_a) and its setpoint is generated by the outer loop; in the outer loop, the controlled variable is the machine speed (n).



While performing the experiment for the motor speed set point (n^*) and armature current (i_a^*), the motor speed (n), the armature current (I_a) and the armature voltage (U_a) will be measured, for various values of the PI controller parameters.

On the target side, the motor speed is regulated using a **dual closed-loop (cascaded) PID control structure**, which is commonly adopted in electric drive systems to improve performance and robustness.

The **outer control loop** is a **speed PID controller**, which compares the speed reference with the measured motor speed and generates a **reference armature current**. This loop is responsible for achieving accurate speed tracking and rejecting load disturbances.

The output of the speed PID serves as the reference for the **inner control loop**, which is a **current PID controller**. This inner loop regulates the actual armature current by adjusting the **duty cycle applied to the inverter IGBT legs**. By controlling the current directly, the system ensures fast torque response and limits current peaks, thereby protecting the power stage and the motor.

The cascaded structure allows the inner current loop to operate at a higher bandwidth than the speed loop, resulting in improved stability and faster transient response. A rate limiter is applied to the duty-cycle command to avoid abrupt changes, ensuring smooth inverter operation and reducing mechanical and electrical stress. The resulting duty cycles are then transmitted to the target and applied to the inverter legs, closing the control loop in real time.

To understand better how communication between the host and target models works, it is advised to carefully review the Host-Target Communication demos in getting started.

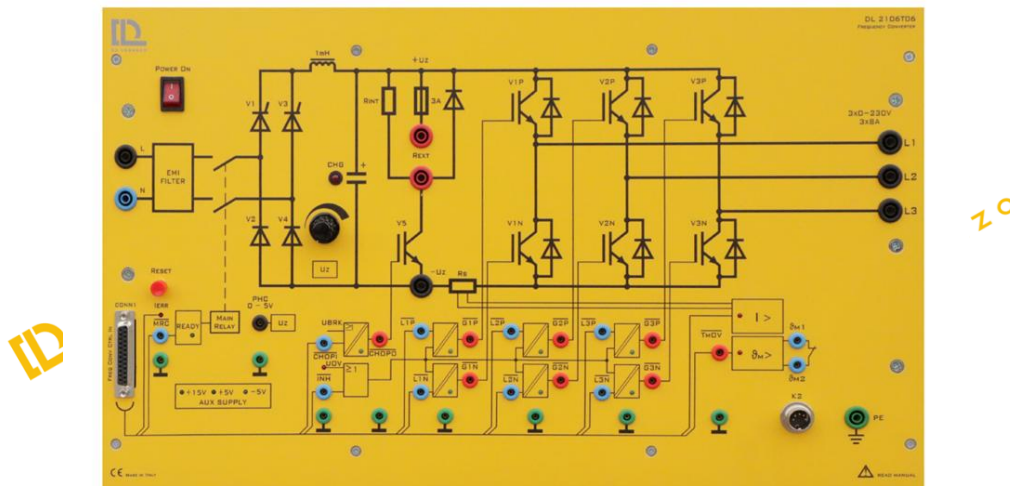
Learning Objectives

By performing this experiment, students will learn about the single-quadrant drive with 1-ph controlled rectifier for dual closed loop speed control, while reaching the following main objectives:

1. Understand how to set up a dc drives with cascade control using the DL 2106T06.
2. Construct master-target models in Simulink, with the Host model dedicated to debugging and monitoring, while the target model executes control algorithms, performs data acquisition, and integrate protection logic with constraints.
3. Implement serial communication between the host and target.
4. To control the output saturation of the DC machine (tune the current loop and the speed loop control).
5. To tune and study the influence of the PID controller parameters on the single-quadrant drive with 1-ph controlled rectifier for dual closed loop speed control.

Part 1: Set up a DC DRIVES WITH CASCADE CONTROL

The DL 2106T06 is a module that integrates a controlled single-phase rectifier, a DC link, and three IGBT legs at the output. The module was originally designed to function as a frequency converter. Nevertheless, owing to the independent controllability of its output inverter, it can also be employed to construct other types of power converters. In this tutorial, we use it as a **DC DRIVES WITH CASCADE CONTROL** for a DC load.



To use the DL 2106T06 frequency converter as a **DC drive**, the system must be reconfigured in the following way:

- **Leg 1** of the output inverter: only the **upper IGBT** (V1P) is controlled using PWM. The lower IGBT (V1N) is forced to be off so it will function as the freewheeling diode of the buck converter.
- **Leg 2**: The lower IGBT (V3N) is forced on to provide a valid return path for the converter current back to the power supply.
- **Leg 3**: both IGBTs are off since this leg is not used in the buck converter.

Part 2: Construct Simulink Host and Target Models

To open the example for reference, run **openDLDemo('dc_drives')** in the command window.

Create a folder for the new project

💡 *It is recommended that each project has its own separate folder, and the folder path does not contain any spaces.*

Navigate the MATLAB working directory to the new project folder

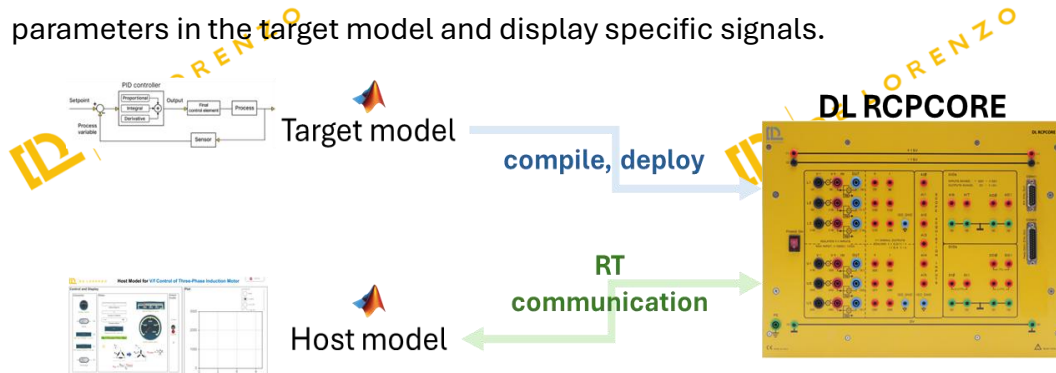
Execute the CreateNew command in the MATLAB Command Window and enter the project name in the pop-up dialog box.

💡 *Avoid using special characters and spaces in the project name.*

Wait until the "Busy" status in the lower-left corner of MATLAB disappears, and no warnings or errors are displayed in the Command Window.

Two models are created by the system:

- **Target model:** This model will be compiled, deployed, and executed on the DL RCP CORE hardware. Its primary responsibility is to execute control algorithms and support real-time communication with the host model.
- **Host model:** This model runs on the PC in simulation mode. Its role is to debug parameters in the target model and display specific signals.



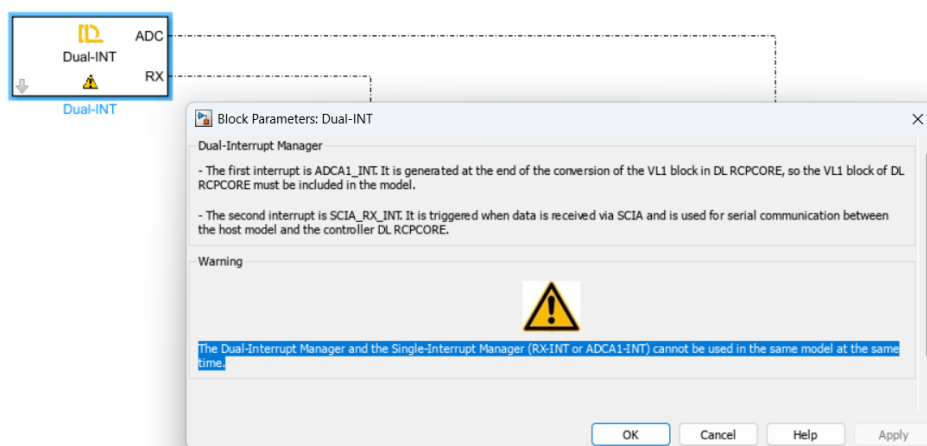
An **.m file** for setting parameters will also be generated automatically.

Step 1: Target Model Structure and Basic Blocks

In the target model, the essential blocks have been added. Read the descriptions and warnings of these blocks to understand their significance and function as essential blocks.

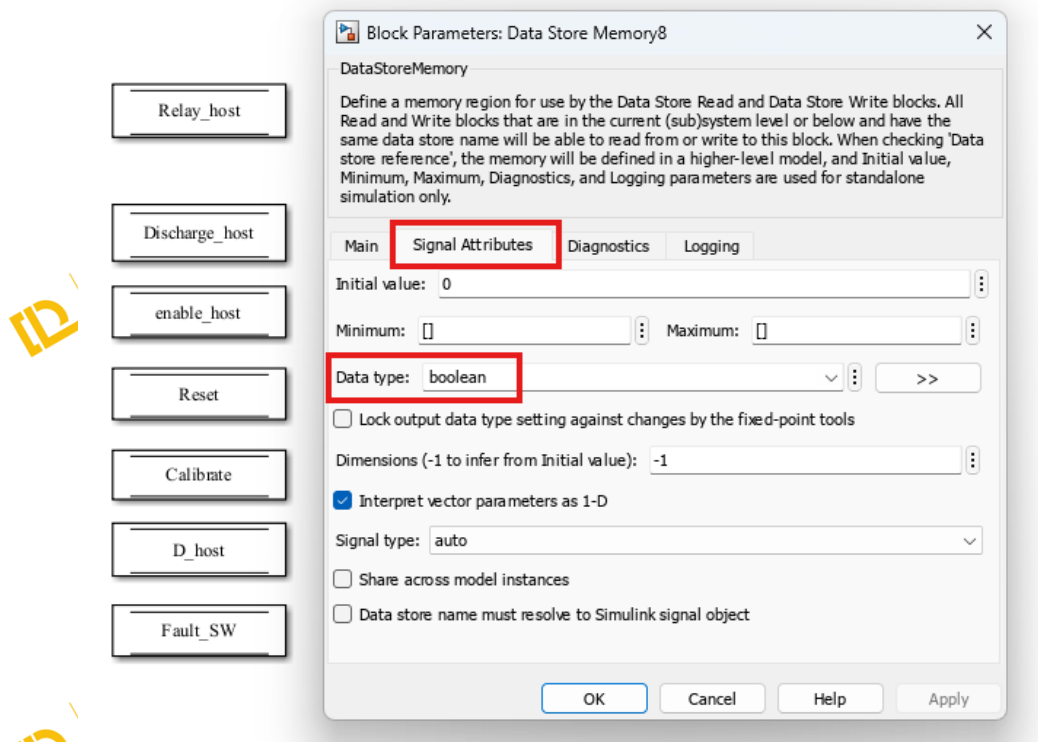
Navigate through the Simulink Library Browser to locate the blocks in the DL library that are required to interface the model with the DL PEL-HIL hardware modules. Add the following blocks:

- **Dual-INT:** The **Dual-Interrupt block** is used to **enable hardware-triggered execution** of two key tasks on the target microcontroller: the **ADC interrupt (ADC)**, which ensures precise and timely execution of ADC conversion and related control routines, and the **RX interrupt (RX)**, which allows efficient reception of serial data only when needed, saving processing resources. By relying on hardware interrupts instead of fixed sample times, the model becomes more reasonable and practical.

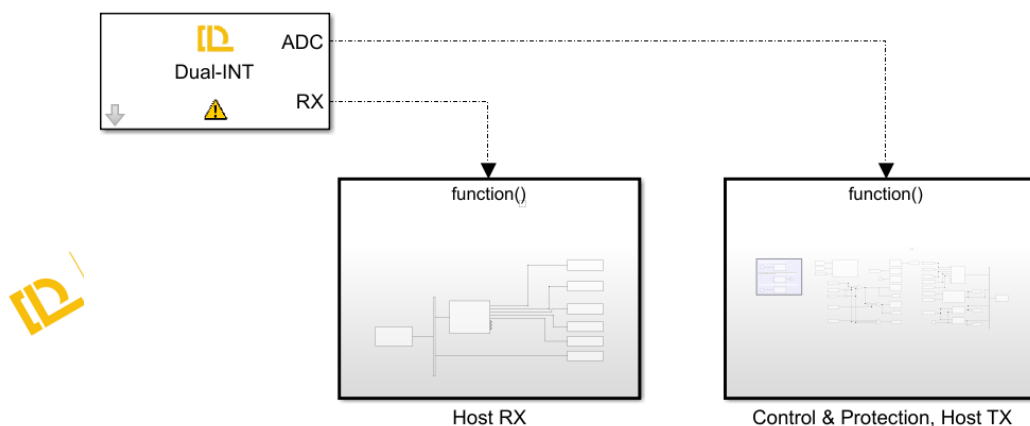


- Preassign key variables in RAM for real-time access using the **Data Store Memory** block.
 - **Relay_host**
Boolean
Relay control signal sent from the host; whether the target follows it depends on the protection algorithm.
 - **Discharge_host**
Boolean
DC link discharge control signal sent from the host; whether the target follows it depends on the protection algorithm.
 - **enable_host**
Boolean
Enable control signal sent from the host for IGBT V1P; whether the target follows it depends on the protection algorithm.
 - **Fault_SW**
Boolean
Based on the results of the software protection algorithm specifically defined for this project. Please distinguish it from hardware fault triggered by hardware protection. The determination of hardware fault is performed entirely by the hardware and follows fixed rules.
 - **Reset**
Boolean
A signal sent by the host to clear latched software faults.
 - **Calibrate**
Boolean
A signal sent by the host to enable hardware data acquisition offset calibration.
 - **V1PN_host**
Single
A signal sent by the host to drive IGBT V1 and V2 with a duty cycle ranging from 0 to 1.

 *Do not forget to specify the Data type of the stored data through Signal Attributes tab, as it is important for efficient and correct communication. Please refer to **Host-Target Communication – Fundamentals** in **DL PEL-HIL Getting Started** to understand the recommended type of data to be used.*

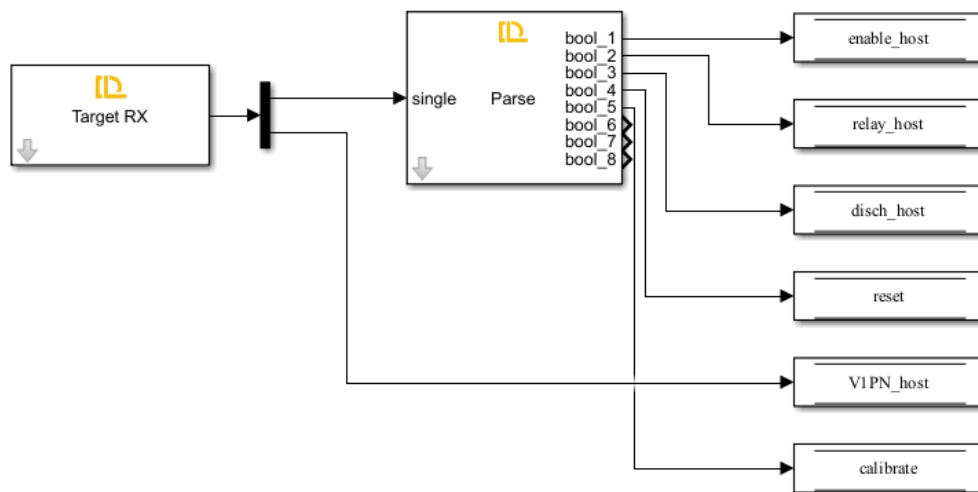


- Create two subsystems and connect them to the two outputs of the Dual-INT block to perform their specified functions as explained before.



The first subsystem is for receiving serial data from the host through Target **RX Block** and writing those data to their specified data store memory blocks to allow them to be read elsewhere in the model.

💡 please refer to *Host-Target Communication – Fundamentals* on DL PEL-HIL Getting Started to understand how the serial communication between host and target works, and what parse blocks is used for.



In this model, the data received from the host include **Relay_host**, **Discharge_host**, **Enable_host**, **Reset**, **Calibrate**, and **V1PN_host**. All of them are stored in the data store memory with their data types specified.

The second subsystem, triggered by the ADC interrupt, performs data acquisition, converter control, protection, and data transmission to the host.

To enable periodic sampling, the modules in the DL Library are designed as follows: using a specific point of the carrier that generates the V1P switching pulse as the reference, the Start of Conversion (SOC) for analog data acquisition is triggered. When the data acquisition of VL1 (of DL RCPCORE) is completed, an interrupt event is generated. This process repeats continuously. Therefore, this subsystem operates at the PWM frequency of V1P.

Ensure V1 or V1P/N block exists if any analog data acquisition block is used.

Ensure VL1 block and the V1 or V1P/N block exist if the Dual-INT or ADCA1-INT block is used.

In this subsystem, add the following blocks from DL library:

- **ctrl en**
Automatically added and configured by the CreateNew command.
This block determines whether the DL RCPCORE control outputs are enabled. It prevents the outputs from being undefined during target model compilation and deployment. The outputs will be enabled only after deployment, so the input of this block is often set to a fixed Boolean value of 1.
- **VL1**
Automatically added and configured by the CreateNew command.

VL1 measures high voltage from channel VL1 of the DL RCPCORE. This block triggers ADCINT1 interrupt and is essential for scheduling with ADC interrupt whether it will be used or not.

- **V1 or V1P/N**

Automatically added and configured by the CreateNew command.

The block controls the 1st IGBT leg of the inverter of DL 2106T06 from the Simulink model by duty cycle ranging from 0 to 1.

It enables the Start of Conversion (SOC) for all other analog measurement blocks. Therefore, it must be included whenever analog signal measurement is required.

To keep leg 1 from being uncontrolled, V1 or V1P/N must be present with a valid duty cycle, even if the leg is not used in the topology.

- **V2 or V2P/N**

Automatically added and configured by the CreateNew command.

The block controls the 2nd IGBT leg of the inverter of DL 2106T06 from the Simulink model by duty cycle ranging from 0 to 1.

To keep leg 2 from being uncontrolled, V2 or V2P/N must be present with a valid duty cycle, even if the leg is not used in the topology.

- **V3 or V3P/N**

Automatically added and configured by the CreateNew command.

The block controls the 3rd IGBT leg of the inverter of DL 2106T06 from the Simulink model by duty cycle ranging from 0 to 1.

To keep leg 3 from being uncontrolled, V3 or V3P/N must be present with a valid duty cycle, even if the leg is not used in the topology.

- **Fault**

This block reads whether the DL 2106T06 has a hardware fault.

- **Relay**

This block controls the input relay of the DL 2106T06.

- **Vdc**

This block acquires the DC link voltage from the DL 2106T06.

- **linv**

This block acquires the DC link output current, i.e., the inverter's supply current.

The control input is determined by both the host-issued **Discharge_host** command and the protection algorithm. When the host initiates DC link discharge to release residual charge, **the target checks the relay state**: the discharge command is passed to the hardware only if the relay is open, preventing overload of the discharge resistor.

As shown here, the Discharge command executed by the target (**Discharge_target**) may differ from the host-issued command (**Discharge_host**), for safety and justified reasons.

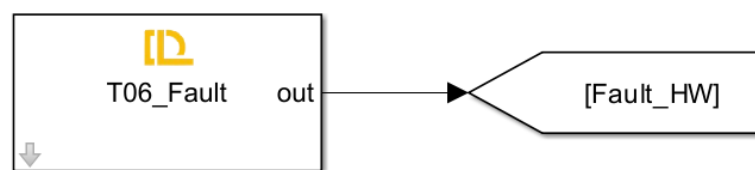
Step 2: Target Model Fault Detection & Protection Logic

1. Fault detection

Faults are classified into two types: hardware-reported DL 2106T06 faults and software-detected faults defined by the user for the application.

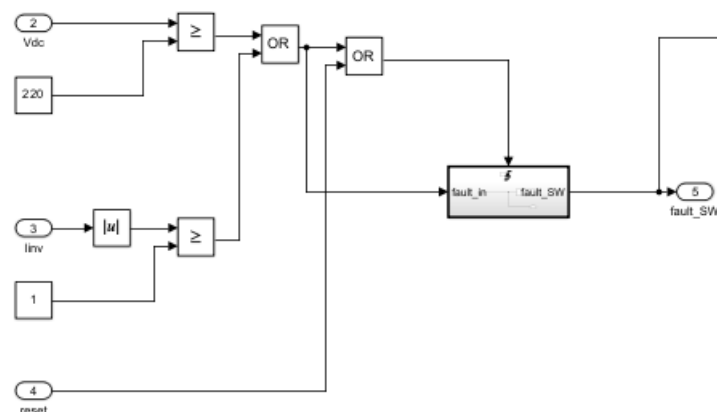
Fault_HW

As mentioned earlier, the Fault block monitors hardware faults in the DL 2106T06.



Fault_SW

To ensure safe operation, a fault is triggered when the DC link exceeds 220 V or the absolute inverter supply current exceeds 1 A. Once a fault occurs, it is latched — meaning the fault remains active even if the conditions return to normal. The fault can only be reset if the user presses the reset button and both the voltage and current fall below their respective thresholds.

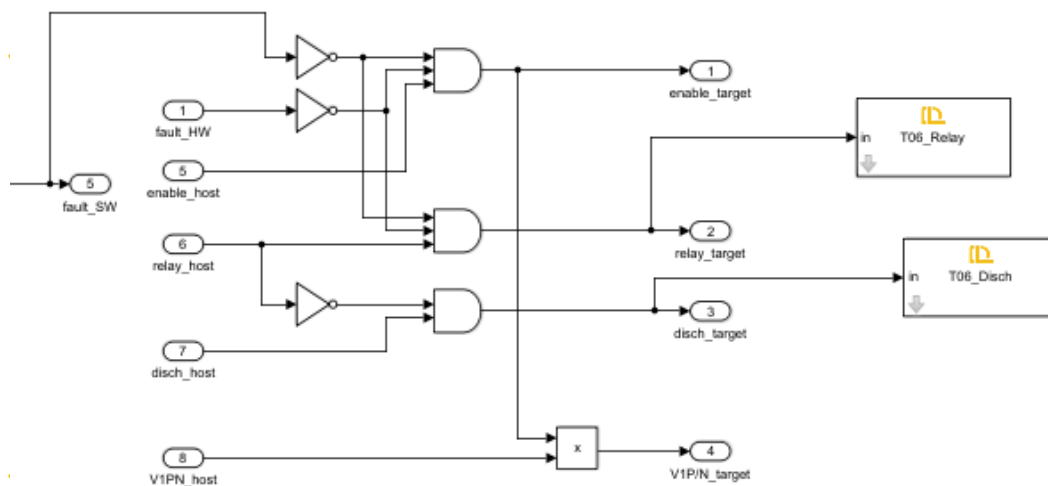


2. Protection Logic

In the target model, several control constraints are implemented to ensure safe and reliable operation of the buck converter. These include:

- The **enable_target** signal is activated only when:
 - **enable_host** = 1 **and**
 - No hardware or software faults are present.

- The **Relay_target** signal (which allows connection of the AC source to the DC link) is set only when:
 - relay_host = 1 **and**
 - No hardware or software faults are present.
- The **discharge_target** signal (used to safely discharge the DC link capacitor) is activated only when:
 - Discharge_host = 1 **and**
 - Relay_target = 0 (ensuring no charging source is connected while discharging to avoid overload).
- The **V1P_target** signal (which controls the upper switch of the first leg, acting as the buck switch) is updated with the value from the host (V1P_host) **only if** enable_target = 1; otherwise, it is forced to zero.

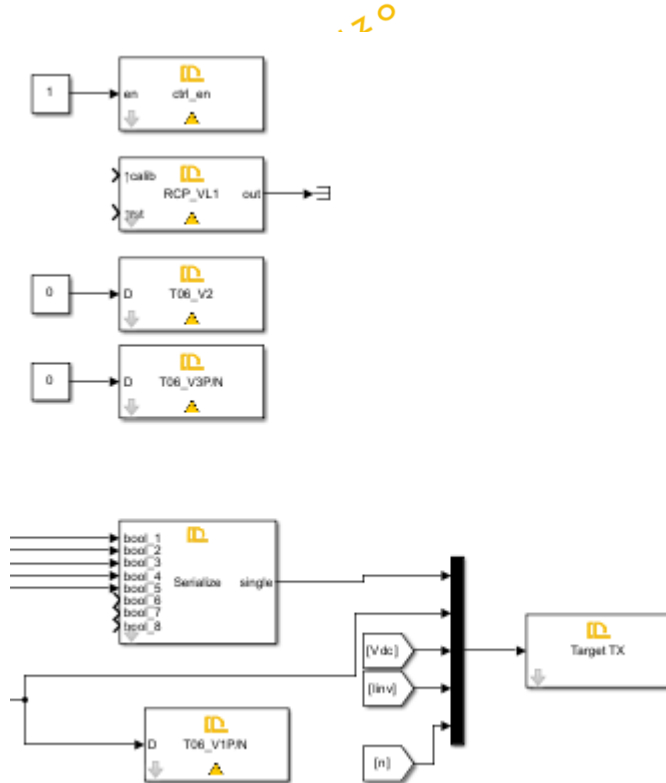


These constraints are essential for the following reasons:

- **Safety:** They prevent power flow or switching commands when a fault is detected, reducing the risk of hardware damage or electrical hazards.
- **Controlled Discharge:** They avoid discharging the capacitor while the relay is still connected to the power supply, which could cause dangerous short circuits.
- **Predictable Behavior:** Ensuring switching only occurs when explicitly allowed by both host and safety logic improves the reliability and traceability of system behavior.

Step 3: Target Model TX

To observe the target's real-time operation in the Host model, signals including relay, DC discharge, control enable, software and hardware faults, Vdc, linv, etc, are transmitted back to the host.



See tutorial 'Host-Target Communication' for details.

Step 4: Host Model

The Host model consists mainly of:

1. Host-Target communication

- RX blocks (to receive status, measurements, fault flags).
- TX blocks (to send commands: Relay_host, Discharge_host, Enable_host, Reset, D_host, etc.).
- Proper framing and timing configuration.

2. Dashboard / GUI

- Switches and knobs:
 - Relay command
 - Discharge command

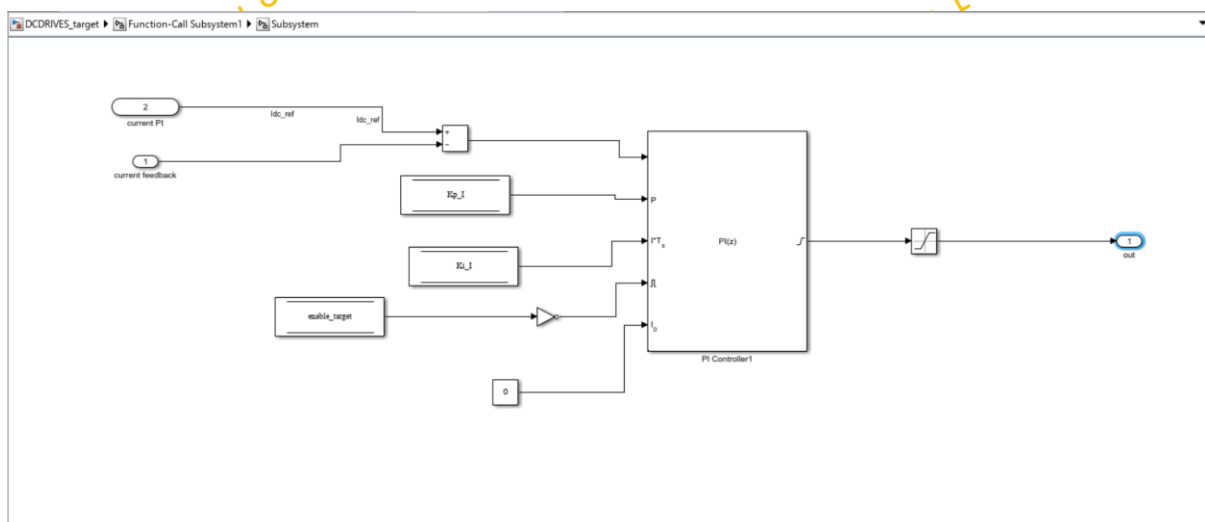
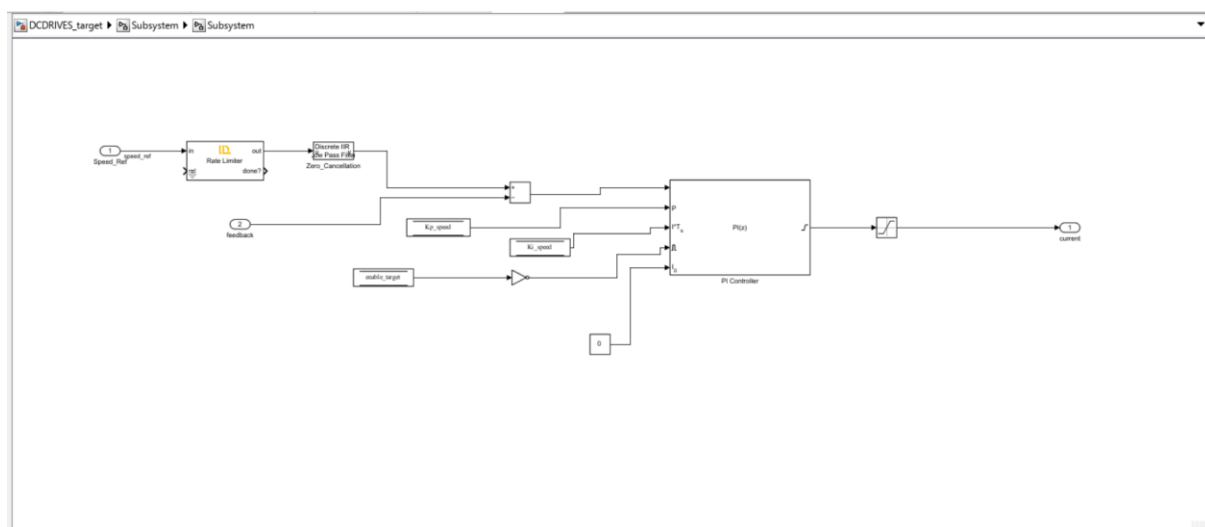
- Enable command
- Reset button
- speed setpoint (slider).
- Displays and scopes:
 - DC link voltage (Vdc)
 - Inverter current (Iinv)
 - Fault indicators
 - Speed value

For details on how to implement the Host side communication and dashboard, refer again to the “**Host-Target Communication**” tutorial.

Step 5: Implement the Cascade Control Algorithm

The previous parts covered the hardware configuration and the model skeleton (interrupts, DataStoreMemory, fault logic, TX/RX communication). This part completes the target model by adding the cascade PI control algorithm: the inner current PI controller (4 kHz), and the outer speed PI controller (400 Hz).

The cascade structure is shown below. The outer speed loop generates a current reference that the inner current loop tracks in real time. Because the inner loop runs 10x faster (4 kHz vs 400 Hz), from the speed loop's perspective the inner loop appears as unity gain — it delivers whatever current is requested within a fraction of the speed-loop sample period.



💡 *Cascade rule: the inner loop must be at least 5–10x faster than the outer loop. In this model the ratio is 10:1 (4 kHz / 400 Hz). Never increase T_{s_speed} below $T_{pwm} * 5$ without also verifying closed-loop stability.*

Step 5.1: Add Data Store Memory Blocks for PI Gains

The PI gains are tunable in real time from the host model. They are stored in DataStoreMemory blocks that both the SCI RX subsystem (write) and the control subsystems (read) can access.

At the top level of the target model, add the following DataStoreMemory blocks:

- **Kp_speed** — Speed PI proportional gain. Data type: single. Initial value: enter "Kp_speed" (the workspace variable name, not 0).
- **Ki_speed** — Speed PI integral gain (per sample). Data type: single. Initial value: "Ki_speed".
- **Kp_I** — Current PI proportional gain. Data type: single. Initial value: "Kp_I".
- **Ki_I** — Current PI integral gain (per sample). Data type: single. Initial value: "Ki_I".
- **n_set** — Speed setpoint from host (RPM). Data type: single. Initial value: "nset".

 **CRITICAL** — Initial Value field: The Initial Value in the Signal Attributes tab must be the workspace variable NAME (e.g. Kp_I).

Also verify that the SCI RX subsystem writes each received value into its corresponding DataStoreMemory block. The receive order must match the transmit order in the host model TX block.

Step 5.2: Current PI Controller (Fast ISR — 4 kHz)

The current PI controller runs inside the ADC interrupt subsystem at 4 kHz (sample time $T_s = T_{\text{pwm}} = 0.25$ ms). Its input is the current reference I_{ref} coming from the speed PI (via Rate Transition), and its output is the duty cycle D sent to the V1P block.

Step 4.2a: Rate Transition Input Block

- Add a Rate Transition block inside the Fast ISR subsystem.
- Set its sample time to T_s (0.25 ms). This block receives the current reference I_{ref} that the 400 Hz speed loop updates every 10 fast samples.
- Label the output I_{ref} . Set data type to single.

Step 4.2b: Current Error

- Add a Sum block with inputs "+ -".

- Connect: I_{ref} to the positive input, $current_{fb}$ to the negative input.
- The output is the current error: $e_I = I_{ref} - current_{fb}$.

Step 5.2c: Discrete PI Block

Use the Simulink Discrete PID Controller block (or the equivalent block from the DL library if provided):

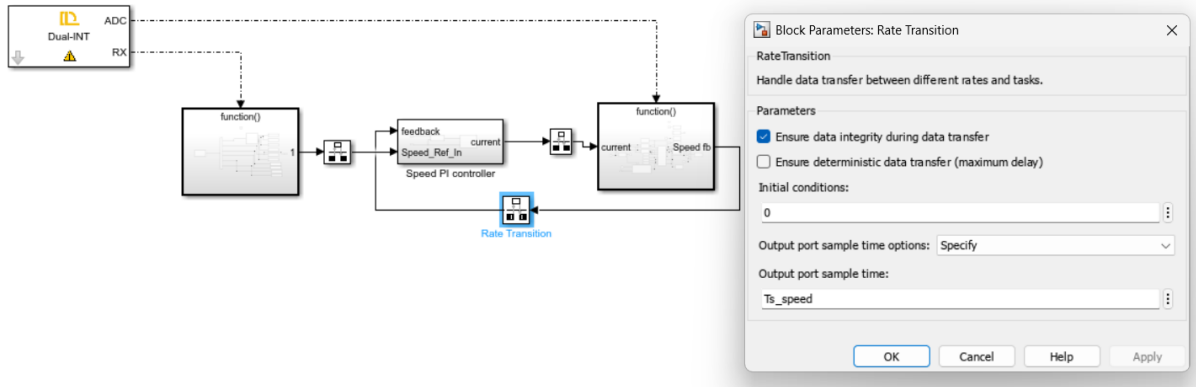
- Controller type: PI
- Discrete method: Forward Euler
- Sample time: T_s (enter as the workspace variable T_s , not a number)
- Proportional gain (P): read from DataStoreMemory Kp_I
- Integral gain (I): read from DataStoreMemory Ki_I
- Anti-windup method: clamp.
- Output saturation: lower limit = 0.0, upper limit = 1.0

 *Ki convention: Ki in this model is a gain per sample, not per second. The block internally computes: $integrator[k+1] = integrator[k] + Ki * e[k]$. Do NOT multiply Ki by T_s inside the block — the sample time is already accounted for in the Ki value set in `DCdrivesparam.m`.*

The output saturation enforces the single-quadrant constraint: duty can only be positive (0 to 1 PU). Negative duty would reverse current, which the freewheeling diode in the buck topology physically prevents.

Step 6: Speed PI Controller (400 Hz Slow Task)

The speed PI runs in a separate subsystem at 400 Hz (sample time $T_{s_speed} = T_{pwm} \times 10 = 2.5$ ms). Create a new subsystem and set its sample time to T_{s_speed} .



Step 6a: Speed Setpoint Conversion

- Add a DataStoreMemory Read block for n_set (speed setpoint in RPM from host).
- Connect its output to a Gain block with value 1/3000 (enter "1/3000").
- This converts the host-sent RPM value to per-unit (PU). For example, 1500 RPM → 0.5 PU.
- Label the output n_set_PU. Set data type to single.

Step 6b: Speed Error

- Add a Sum block with inputs "+ -".
- Connect: n_set to the positive input, speed_fb to the negative input.

Step 6c: Discrete PI Block

- Add a Discrete PID Controller block.
- Controller type: PI
- Discrete method: Forward Euler
- Sample time: Ts_speed
- Proportional gain (P): read from DataStoreMemory Kp_speed
- Integral gain (I): read from DataStoreMemory Ki_speed
- Anti-windup method: clamp
- Output saturation: lower limit = -1.5, upper limit = +1.5

The output of the speed PI is the current reference I_{ref} . A value of +1.5 PU means "request 100% of rated current for maximum acceleration." A value of -1.5PU requests

braking current — however, since the current PI lower saturation is 0.0, negative current references are silently clamped to zero.

Step 5d: Rate Transition Output Block

- Add a Rate Transition block at the output of the speed PI.
- This block transfers the current reference I_{ref} from the 400 Hz domain to the 4 kHz domain.
- Connect this Rate Transition output to the Rate Transition input block inside the Fast ISR subsystem (Step 8a).

 In Simulink, right-click both Rate Transition blocks and verify that the sample times are set correctly: the output-side Rate Transition should have sample time T_{s_speed} , and the input-side Rate Transition inside the fast ISR should have sample time T_s . The Rate Transition block automatically handles the data transfer between the two rates.

Step 7: Set Up the Parameter File (DCdrivesparam.m)

Before building the model, run DCdrivesparam.m to define all workspace variables. The complete file should contain:

```
%% DC DRIVES WITH CASCADE CONTROL – Parameter File
```

```
%% Timing
```

```
PWM_frequency = 4e3;           % Hz – PWM switching frequency
T_pwm         = 1/PWM_frequency; % 0.25 ms
Ts            = T_pwm;          % Fast ISR sample time (current loop)
Ts_speed      = T_pwm * 10;     % 2.5 ms (speed loop)
dataType      = 'single';
```

```
%% Hardware
```

```
target      = SetControllerDetails('DL RCPCORE', PWM_frequency, deadtime);
target.comport = 'COM8';          % Update to your serial port
converter    = SetConverterParameters('DL 2106T06');
```

```
%% Host control signals
```

```
stop        = boolean(0);
```

```

enable_host    = boolean(0);
relay_host     = boolean(0);
disch_host     = boolean(0);
nset           = single(0);      % Speed setpoint in RPM

%% PI Gains (starting values – tune on hardware)
Kp_speed = single(0.05);        % Speed PI proportional gain
Ki_speed = single(0.002);       % Speed PI integral gain (per sample)
Kp_I      = single(0.08);       % Current PI proportional gain
Ki_I      = single(0.003);      % Current PI integral gain (per sample)

```

💡 *Ki is a per-sample gain. Equivalent continuous integral gain = K_i / T_s . For $K_{i_I} = 0.003$: $K_{i_continuous} = 0.003 / 0.00025 = 12/s$. Do NOT multiply K_i by T_s inside the Simulink block — it is already included.*

Step 8: Connect the SCI RX Subsystem to PI Gain DataStore Blocks

The host model sends the four PI gains (K_p_speed , K_i_speed , K_p_I , K_i_I) via serial communication. The SCI RX subsystem must write each received value into the corresponding DataStoreMemory block.

Inside the SCI RX subsystem, after the Target RX and Parse blocks:

- Connect the parsed K_p_speed value to a DataStoreMemory Write block for K_p_speed .
- Connect K_i_speed to its DataStoreMemory Write block.
- Connect K_p_I to its DataStoreMemory Write block.
- Connect K_i_I to its DataStoreMemory Write block.
- The receive channel index must match the transmit channel index in the host model TX block. Verify the channel order: [Relay, Discharge, Enable, Reset, Calibrate, V1PN, K_p_speed , K_i_speed , K_p_I , K_i_I , nset].

💡 *To verify the channel mapping: in the host TX block, hover over each input port to see its channel number. In the target RX block, the parsed output at index N corresponds to channel N in the host TX block. A mismatch here means the wrong gain is written to the wrong controller.*

💡 Add four Edit Field blocks for K_p_speed , K_i_speed , K_p_I , K_i_I to allow real-time gain adjustment without rebuilding the model. Connect each Edit Field output to the host TX block at the appropriate channel. This allows PI tuning while the drive is running.

Step 9: Target Model Build, Deploy and Start

Step 9a: Final Pre-Build Checklist

- DCdrivesparam.m has been run in MATLAB. Verify by typing "Kp_I" in the Command Window — it should return 0.08 (or your chosen starting value).
- All DataStoreMemory Initial Value fields contain workspace variable names, not 0.
- The speed PI subsystem sample time matches Ts_speed (not Ts).
- The V1P duty path contains current PI $\rightarrow$ saturation $[0,1] \rightarrow$ V1P block. No direct constant is connected to V1P.
- Both Rate Transition blocks are present and have matching signal names.
- The SCI RX subsystem writes all received signals (including PI gains) to their DataStoreMemory blocks.

Step 9b: Build and Deploy

- Save the target model (Ctrl+S).
- In MATLAB, run DCdrivesparam.m to ensure all workspace variables are up to date.
- In Simulink, go to Apps > Embedded Coder (or use the Hardware tab) and click Build, Deploy & Start.
- Wait for the build to complete. Check the MATLAB Command Window for errors. Common errors: undefined variable (DCdrivesparam.m not run), sample time mismatch (Ts vs Ts_speed confusion).

Step 9c: First Power-On Sequence

Follow this exact sequence every time the drive is powered on:

- Step 1: Run the host model in Simulink (press Play).
- Step 2: Verify V_{dc} reads the expected DC link voltage (~ 150 V). Verify $current_fb$ reads approximately 0 A (within offset). Verify $speed_fb$ reads 0 RPM.

- Step 3: Enable the relay (set `relay_host = 1`). Verify relay status indicator in host turns green.
- Step 4: Enable the inverter gate (set `enable_host = 1`).
- Step 5: Set `n_set` to a low value (e.g. 200 RPM) and observe the motor response.
- Step 6: If no faults are triggered and `speed_fb` rises toward 200 RPM, the cascade control is working.

Step 10: Verification and Acceptance Checks

Once the model is deployed and PI gains are tuned, verify the following:

- **Speed tracking:** Apply step commands at 300, 600, 900, 1200, and 1500 RPM. `Speed_fb` should follow `n_set` with overshoot $< 20\%$ and settle within 3–5 seconds at each step.
- **Current limiting:** During acceleration, `current_fb` should not exceed 1.5. If it does, reduce the speed PI gains or lower the current PI upper saturation limit.
- **Fault protection:** Verify that a hardware fault (`Fault_HW`) immediately disables the inverter output. Verify the software fault threshold ($|I_{inv}| > 2\text{ A}$ or $|V_{dc}| > 220\text{ V}$) triggers `Fault_SW` and latches it until Reset is applied.
- **Deceleration:** Step from 1500 RPM down to 300 RPM and verify the motor decelerates without oscillation. The rate limiter on `n_set` should be active for this test.
- **Zero setpoint:** Set `n_set = 0` RPM and verify the motor comes to a complete stop within a reasonable time (dependent on load inertia and friction). `Speed_fb` should settle at 0, not reverse.

💡 To confirm the cascade structure is working correctly: watch both `speed_fb` and `current_fb` during a speed step. You should see `current_fb` spike briefly (inner loop responding to the large current reference from the speed PI) and then settle to a low steady-state value once the target speed is reached. This current spike $\rightarrow$ settle pattern is the mark of correct cascade behaviour.